Project Report on

AI Resume Screening System with Tracing

SUBMITTED BY: ATMAKURI NIKITHA

1. Introduction

Recruitment processes often involve reviewing a large number of resumes, which is time-consuming and prone to inconsistency. To address this challenge, this project presents an **AI Resume Screening System** that automates candidate evaluation using a structured pipeline.

The system analyses resumes, compares them with a job description, assigns a suitability score, and provides a clear explanation. It is designed using **angChain** and includes **LangSmith tracing** for monitoring and debugging.

2. Objective

The objectives of this project are:

- To develop an automated resume screening system
- To extract relevant information such as skills, experience, and tools
- To compare candidate profiles with job requirements
- To assign a score representing candidate suitability
- To generate explainable outputs
- To implement tracing for debugging and monitoring

3. System Architecture

The system follows a modular pipeline architecture, where each stage performs a specific task.

Resume → Extraction → Matching → Scoring → Explanation

Each stage is implemented as an independent module, allowing easy modification and debugging.

---

4. Pipeline Description

The system consists of four main stages:

4.1 Extraction Stage

This stage extracts key information from the resume, including:

- Name
- Skills
- Experience
- Tools

The extraction process ensures that only explicitly mentioned details are considered.

4.2 Matching Stage

In this stage, the extracted data is compared with job requirements. It identifies:

- Matched skills, Missing skills, Experience level, Relevant tools

4.3 Scoring Stage

A score between **0 and 100** is assigned based on:

Number of matched skills, Experience level, Missing requirements

The candidate is categorized as:

- **Strong** (80–100)
- **Average** (50–79)
- **Weak** (0–49)

4.4 Explanation Stage

This stage generates a human-readable explanation that justifies the assigned score based on:

- Skill match
- Missing requirements
- Experience evaluation

---

5. Prompt Engineering Approach

Initially, prompt-based techniques using language models were explored to perform extraction and evaluation. The prompts included rules such as:

- Do not assume missing skills
- Extract only required fields
- Avoid hallucination

However, challenges such as inconsistent outputs and repetition were observed. To overcome this, the system was redesigned using **rule-based logic and structured outputs**, ensuring reliability and accuracy.

---

6. Tools and Technologies

The following technologies were used:

- **Python** for implementation
- **LangChain** for building modular pipelines
- **LangSmith** for tracing and debugging
- **VS Code** as development environment

---

7. Implementation Details

The system processes three types of resumes:

- Strong candidate
- Average candidate
- Weak candidate

Along with a job description.

Example Extracted Output

Nam : Atmakuri Nikitha

Skil : Python, Machine Learning, SQL

Exper *: Worked on NLP projects*

T : TensorFlow, PyTorch

Each resume is passed through all pipeline stages, and results are displayed in a structured format.

---

8. LangChain Implementation

The system is implemented using **LangChain's Runnable architecture**, where:

- Each stage is a separate chain
- Chains are connected sequentially
- .invoke() method is used for execution

This modular design improves maintainability and clarity.

---

9. LangSmith Tracing

LangSmith is used to monitor the pipeline execution. It provides:

- Step-by-step tracking of each stage
- Visibility into inputs and outputs
- Debugging support for incorrect results

Each candidate evaluation is recorded as a separate run, ensuring transparency in the process.

---

10. Result Analysis

The system successfully evaluated all candidates:

- **Strong Candidate** showed high skill alignment and received a high score
- **Average Candidate** showed moderate alignment and received a medium score
- **Weak Candidate** showed low alignment and received a low score

The outputs were consistent, structured, and explainable.

---

11. Challenges Faced

Several challenges were encountered during development:

- ***Model & API Issues***: Faced errors like *model not supported*, *503 server error*, and pipeline mismatch.
- ***Repetitive and Incorrect Outputs***: Model was repeating prompts and giving inconsistent results.

- ***Token Length and Input Limit Problems:*** Long resumes caused token limit warnings and failures.
- ***Unstructured Output Format****:* Output was messy and not suitable for analysis or tracing

These were resolved by:

- Removed dependency on unstable APIs and switched to a reliable rule-based approach.
- Replaced LLM-based generation with deterministic logic for accurate outputs.
- Reduced input size and later avoided model-based processing.
- Used structured dictionary outputs, enabling clear display in LangSmith respectively.

---

12. Advantages:

- Automates resume screening efficiently
- Provides consistent and unbiased evaluation and generates explainable results
- Modular and scalable design
- Easy debugging using LangSmith

---

13. Conclusion

This project demonstrates the development of a **modular AI pipeline** for resume screening using LangChain. It highlights the importance of structured system design, explainability, and debugging in building reliable AI applications.

The system successfully automates candidate evaluation and provides meaningful insights, making it useful for real-world recruitment scenarios.

---

14. Final Insight

This project provided hands-on experience in:

- Designing AI pipelines, implementing modular architectures and debugging and monitoring using LangSmith

It serves as a strong foundation for building real-world applications in **Generative AI and LLM engineering**